

L A B 3

SQL Joins

A beginner-friendly introduction to combining tables

Ioannis Tsioulis

i.tsioulis@mc-class.gr

JOIN

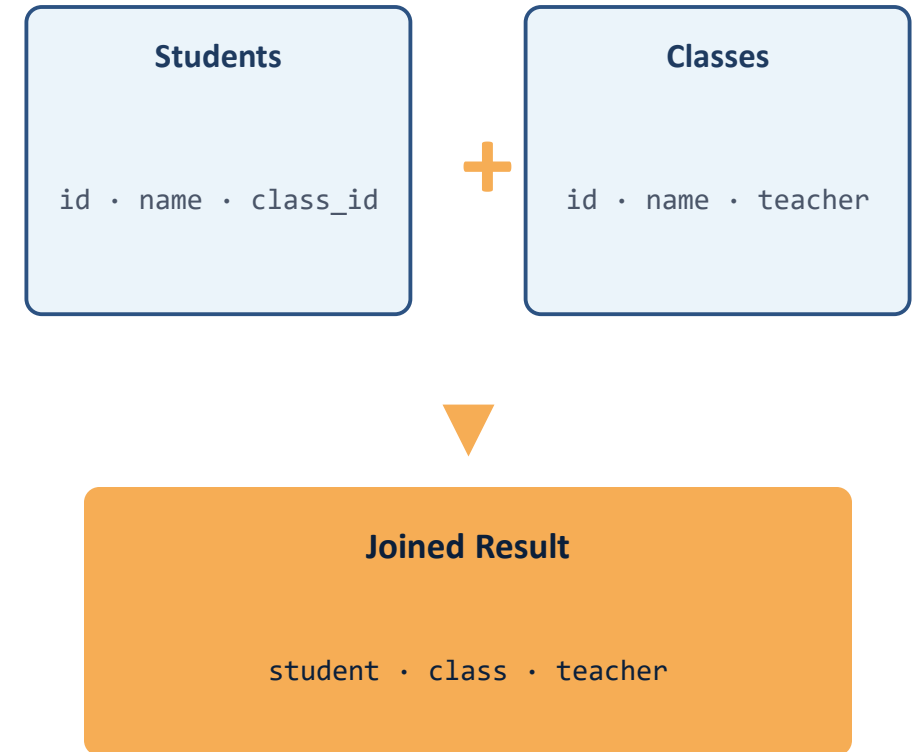
What is a JOIN?

Bringing related data back together

In a relational database, data is usually split across multiple tables to avoid repetition.

A **JOIN** is an SQL operation that combines rows from two or more tables based on a *related column* between them.

Think of it as matching puzzle pieces — data lives in separate tables, and joins let you bring it together into one unified result.



Why Do We Need Joins?

A quick real-world analogy

Imagine a school system.

✗ ONE GIANT TABLE

Student + class info in a single table.

Every student row repeats:

- class name
- teacher name
- classroom

Redundant, error-prone, hard to update.

✓ SEPARATE TABLES + JOIN

Students in one table, classes in another.

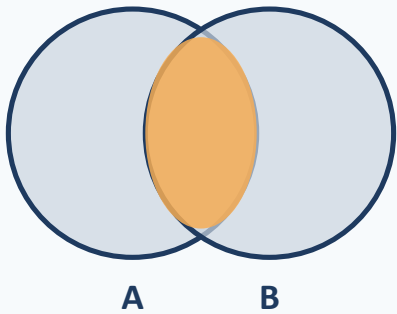
Link them with a shared column:
class_id

Use a JOIN when you need the full picture.

No repetition, easy to update.

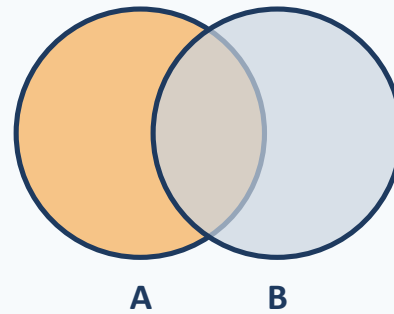
The Four Main Types of Joins

Each one returns a different slice of the data



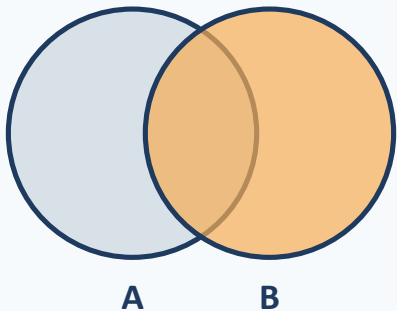
INNER JOIN

Only rows that match in BOTH tables.



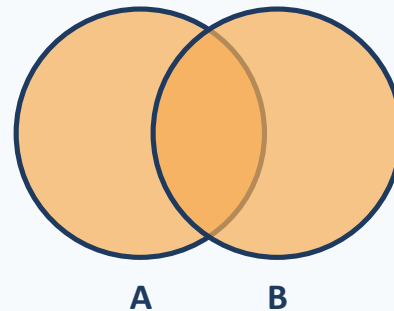
LEFT JOIN

ALL rows from the left + matches from the right.



RIGHT JOIN

ALL rows from the right + matches from the left.



FULL OUTER JOIN

ALL rows from both tables.

Basic JOIN Syntax

The skeleton every JOIN query follows

syntax.sql

```
SELECT columns
FROM table1
INNER JOIN table2
    ON table1.common_column = table2.common_column;
```

FROM

The starting (left) table

JOIN

The table you attach to it

ON

How they match (shared column)

Our Example Tables

We'll use these throughout the lab

Students

student_id	name	class_id
1	Alice	10
2	Bob	20
3	Carol	10
4	David	NULL
5	Eve	30

Classes

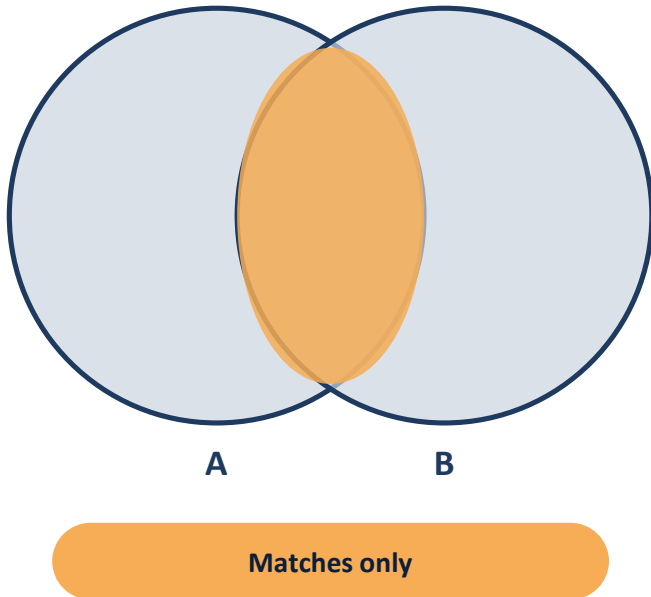
class_id	class_name
10	Mathematics
20	History
40	Physics

Notice the mismatches (highlighted in amber):

- David has NULL class
- Eve's class 30 does not exist
- Physics (40) has no students

INNER JOIN

Only rows that match in both tables



inner_join.sql

```
SELECT s.name, c.class_name
FROM Students AS s
INNER JOIN Classes AS c
    ON s.class_id = c.class_id;
```

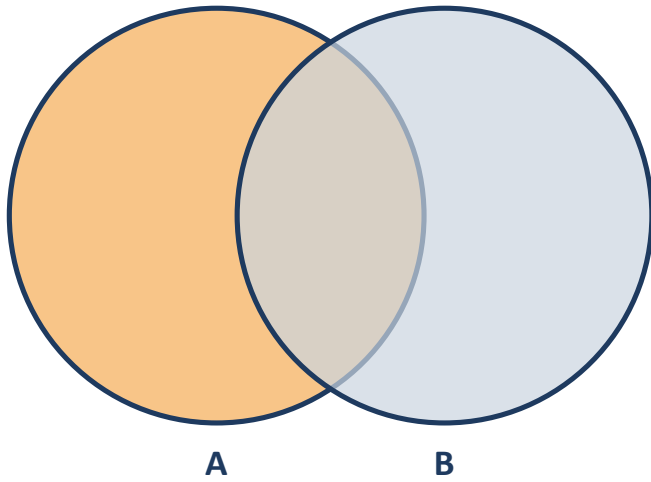
Result: 3 rows

name	class_name
Alice	Mathematics
Bob	History
Carol	Mathematics

David (NULL class) and Eve (class 30 not in Classes) are excluded.

LEFT JOIN

All rows from the left + matches from the right



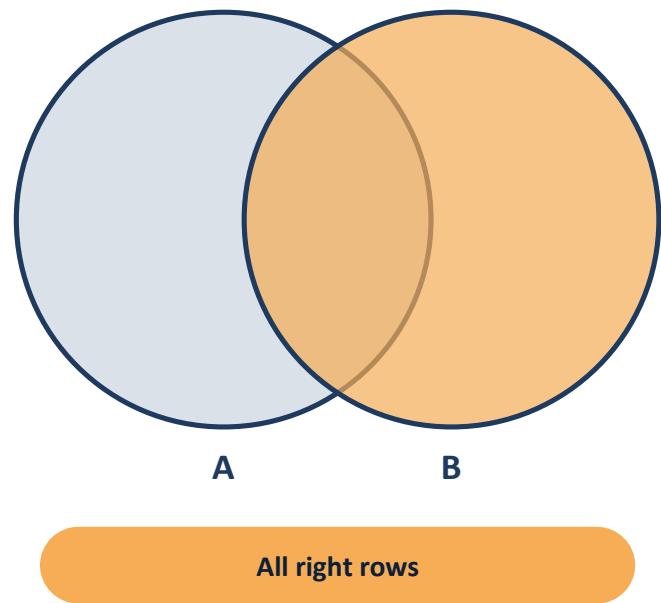
```
SELECT s.name, c.class_name
FROM Students AS s
LEFT JOIN Classes AS c
  ON s.class_id = c.class_id;
```

Result: all 5 students

name	class_name
Alice	Mathematics
Bob	History
Carol	Mathematics
David	NULL
Eve	NULL

RIGHT JOIN

All rows from the right + matches from the left



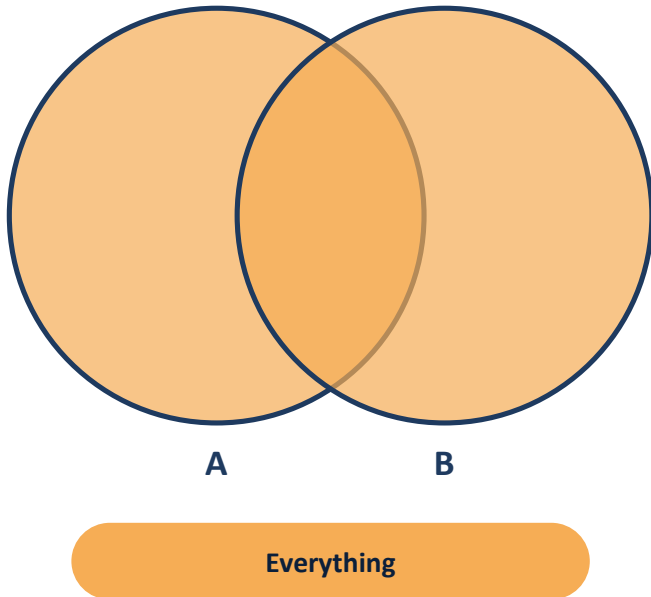
```
SELECT s.name, c.class_name
FROM Students AS s
RIGHT JOIN Classes AS c
ON s.class_id = c.class_id;
```

Result: one row per class match + empty classes

name	class_name
Alice	Mathematics
Carol	Mathematics
Bob	History
NULL	Physics

FULL OUTER JOIN

All rows from both tables



```
full_outer.sql

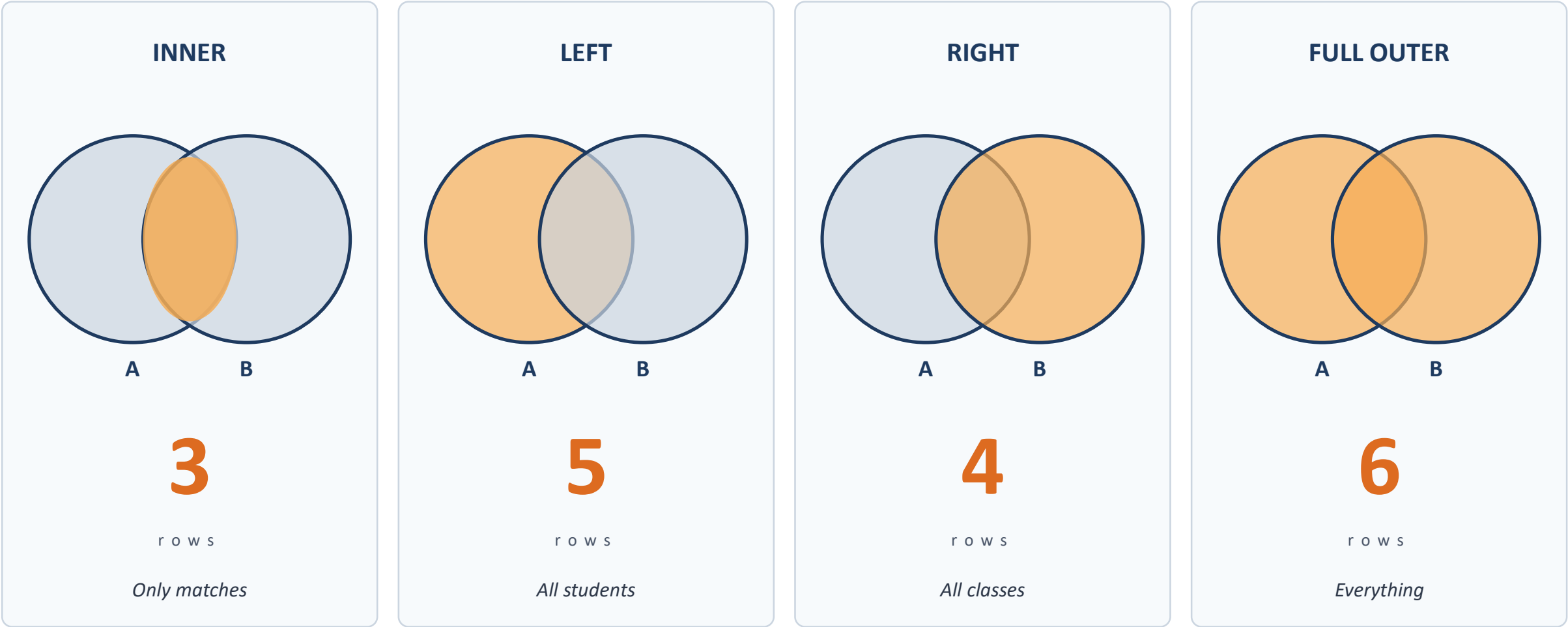
SELECT s.name, c.class_name
FROM Students AS s
FULL OUTER JOIN Classes AS c
  ON s.class_id = c.class_id;
```

Result: every student and every class

name	class_name
Alice	Mathematics
Bob	History
Carol	Mathematics
David	NULL
Eve	NULL
NULL	Physics

At a Glance: All Four Joins

Same data, four different views



Common Pitfalls

Watch out for these beginner mistakes

1

Forgetting the ON clause

Without ON, many databases fall back to a CROSS JOIN — a massive Cartesian product. Always match on a column.

3

Counting * with LEFT JOIN

COUNT(*) counts every output row (including unmatched rows). To count real matches, use COUNT(right_table.id) so unmatched rows return 0.

2

Confusing LEFT with RIGHT

The 'left' table is the one right after FROM. When in doubt, swap the tables and use LEFT JOIN — it reads more naturally.

4

Ambiguous column names

When both tables have a column called id or name, you must qualify it: students.name, not just name. Aliases (s, c) make queries much cleaner.

Practice & Resources

Where to go next

TIME TO PRACTICE

Open the Lab 3 exercises

10 exercises

Copy the CREATE TABLE & INSERT code from the lab PDF, then work through each exercise.

Run everything here:

programiz.com/sql/online-compiler

Check your answers on the last page of the answers PDF.

FURTHER READING

W3Schools – SQL Joins

Beginner-friendly, with "Try it Yourself"

w3schools.com/sql/sql_join.asp

Coding Horror – A Visual Explanation

The classic Venn diagram article by Jeff Atwood

blog.codinghorror.com/a-visual-explanation-of-sql-joins/

LearnSQL – JOINS Made Easy

Fun example with artists & songs

learnsql.com/blog/sql-joins-made-easy/

Questions?

Let's practice what we just learned.

Ioannis Tsioulis

i.tsioulis@mc-class.gr